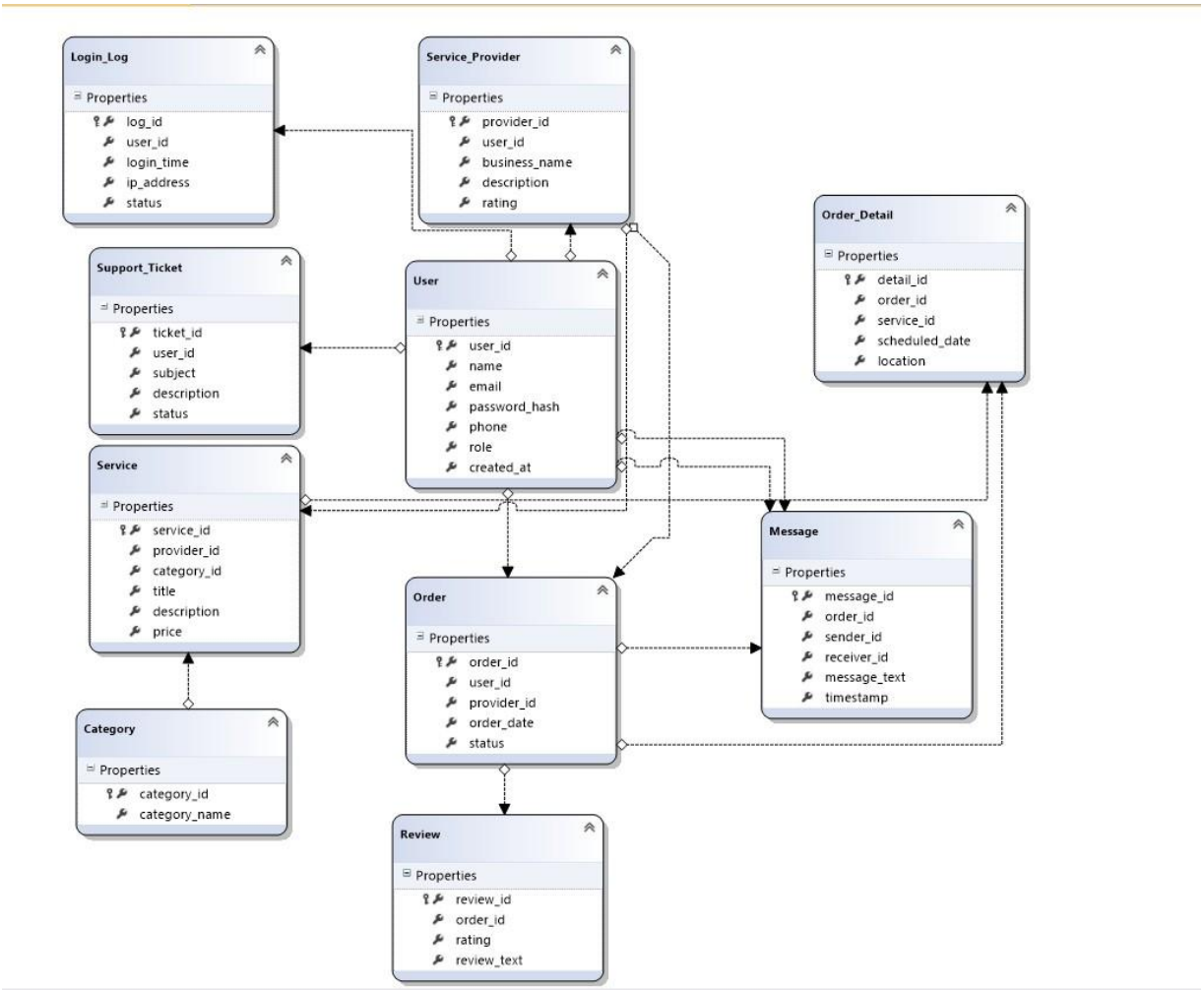
1. Physical Database Creation

Students must implement the physical database in MySQL/SQL Server. This includes:

- CREATE DATABASE
- CREATE TABLE statements
- Defining relationships using FK constraints



```sql
CREATE TABLE Users (
    user_id INT IDENTITY PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100) UNIQUE,
    password_hash VARCHAR(255),
    phone VARCHAR(20),
    role VARCHAR(20), -- Customer / Provider / Admin
    created_at DATETIME DEFAULT GETDATE()
);

CREATE TABLE Service_Providers (
    provider_id INT IDENTITY PRIMARY KEY,
    user_id INT FOREIGN KEY REFERENCES Users(user_id),
    business_name VARCHAR(100),
    description VARCHAR(500),
    rating DECIMAL(3,1)
);

CREATE TABLE Categories(
    category_id INT IDENTITY PRIMARY KEY,
    category_name VARCHAR(100)
);

CREATE TABLE Services(
    service_id INT IDENTITY PRIMARY KEY,
    provider_id INT FOREIGN KEY REFERENCES Service_Providers(provider_id),
    category_id INT FOREIGN KEY REFERENCES Categories(category_id),
    title VARCHAR(100),
    description VARCHAR(500),
    price DECIMAL(10,2)
);
```

```sql
CREATE TABLE Orders(
    order_id INT IDENTITY PRIMARY KEY,
    user_id INT FOREIGN KEY REFERENCES Users(user_id),
    provider_id INT FOREIGN KEY REFERENCES Service_Providers(provider_id),
    order_date DATETIME DEFAULT GETDATE(),
    status VARCHAR(50)
);

CREATE TABLE Order_Details(
    detail_id INT IDENTITY PRIMARY KEY,
    order_id INT FOREIGN KEY REFERENCES Orders(order_id),
    service_id INT FOREIGN KEY REFERENCES Services(service_id),
    scheduled_date DATETIME,
    location VARCHAR(200)
);

CREATE TABLE Messages(
    message_id INT IDENTITY PRIMARY KEY,
    order_id INT FOREIGN KEY REFERENCES Orders(order_id),
    sender_id INT FOREIGN KEY REFERENCES Users(user_id),
    receiver_id INT FOREIGN KEY REFERENCES Users(user_id),
    message_text VARCHAR(MAX),
    timestamp DATETIME DEFAULT GETDATE()
);

CREATE TABLE Reviews(
    review_id INT IDENTITY PRIMARY KEY,
    order_id INT FOREIGN KEY REFERENCES Orders(order_id),
    rating INT,
    review_text VARCHAR(MAX)
);
```

```sql
CREATE TABLE Support_Tickets(
    ticket_id INT IDENTITY PRIMARY KEY,
    user_id INT FOREIGN KEY REFERENCES Users(user_id),
    subject VARCHAR(100),
    description VARCHAR(MAX),
    status VARCHAR(50)
);

CREATE TABLE Login_Logs(
    log_id INT IDENTITY PRIMARY KEY,
    user_id INT FOREIGN KEY REFERENCES Users(user_id),
    login_time DATETIME DEFAULT GETDATE(),
    ip_address VARCHAR(50),
    status VARCHAR(50)
);
```

2. Data Insertion

Students must insert sample data representing realistic operations:

• Admin user

• At least 2 other users with different roles

• Domain-specific data (books, patients, items, etc.)

NOTE: Make sure each tables has at least 10 records and does not violate entity integrity or referential integrity constraints.

dbo.Users [Data] | ThrivePortal.dbml | Login.aspx | ThrivePortal.dbml

Max Rows: 1000

user_id	name	email	password_hash	phone	role	created_at
1	Taqwa	taqwa@gmail.c...	tq21	123456	Admin	02/12/2025 12:...
5	Rohail	rohail1@gmail....	rm06	123455	ServiceProvider	02/12/2025 12:...
6	Tehreem	tehreem2@gm...	tm19	123444	Customer	02/12/2025 12:...
7	Aiman	aiman3@gmail....	am03	122222	Customer	10/12/2025 9:4...
NULL	NULL	NULL	NULL	NULL	NULL	NULL

- Categories
- Login_Logs
- Messages
- Order_Details
- Orders
- Reviews
- Service_Providers
- Services
- Support_Tickets
- Users

dbo.Categories [Data] | dbo.Service_Providers

Max Rows: 1000

category_id	category_name
1	Home Services
2	Tutoring & Edu...
3	Health & Welln...
4	IT & Tech Supp...
5	Event PLanning
6	Car Wash and D...
7	Errand Service
8	Photography an...
9	Catering and M...
10	Laundry and Iro...

3. User and Role Management

Students must implement:

• User accounts

• Role creation

• Privilege assignment

• Mapping of users to roles

dbo.Users [Data] | ThrivePortal.dbml | Login.aspx | ThrivePortal.dbml

Max Rows: 1000

user_id	name	email	password_hash	phone	role	created_at
1	Taqwa	taqwa@gmail.c...	tq21	123456	Admin	02/12/2025 12:...
5	Rohail	rohail1@gmail....	rm06	123455	ServiceProvider	02/12/2025 12:...
6	Tehreem	tehreem2@gm...	tm19	123444	Customer	02/12/2025 12:...
7	Aiman	aiman3@gmail....	am03	122222	Customer	10/12/2025 9:4...
NULL	NULL	NULL	NULL	NULL	NULL	NULL

4. Data Retrieval (Queries)

Students must write SQL queries involving:

• Joins (INNER, LEFT, RIGHT)

• Aggregate functions

• Conditions based on user roles

Example: Managers may retrieve summary reports, while standard users can only retrieve their own data.

```
1 reference
private void LoadOrders()
{
    try
    {
        // Fetch orders along with customer, provider, and service
        var orders = from o in db.Orders
                     join u in db.Users on o.user_id equals u.user_id
                     join p in db.Service_Providers on o.provider_id equals p.provider_id
                     join s in db.Services on p.provider_id equals s.provider_id
                     select new
                     {
                         o.order_id,
                         user_name = u.name,
                         provider_name = p.business_name,
                         service_title = s.title,
                         o.order_date,
                         o.status
                     };
```

5. Interface Development

Each role must have a dedicated interface (web or desktop) based on its privileges:

• Admin Interface: full system management

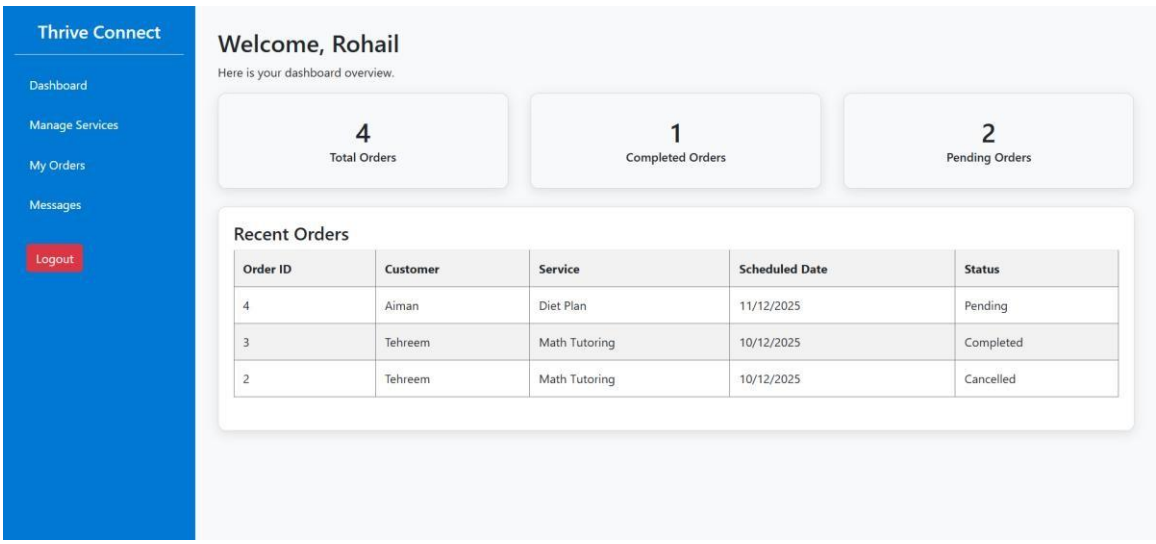
• Role 1 Interface: limited access

• Role 2 Interface: minimum access

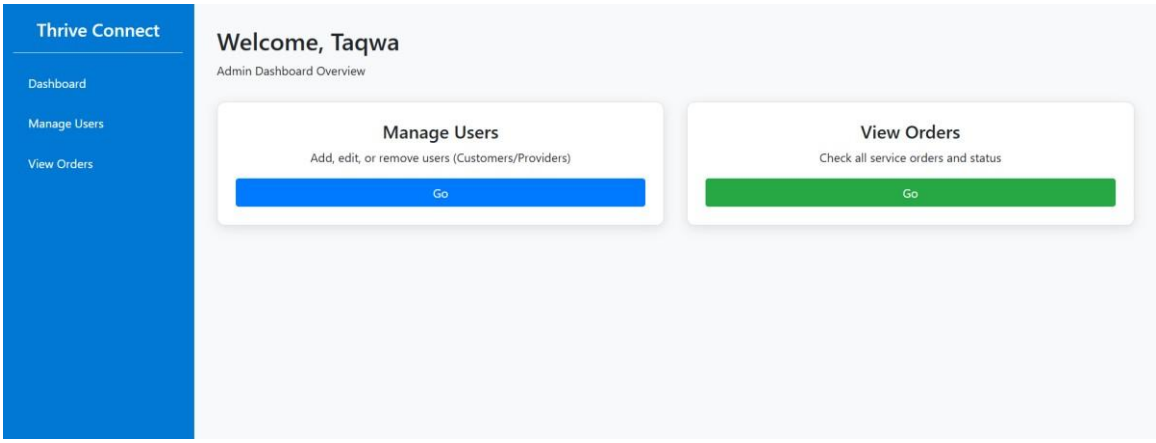
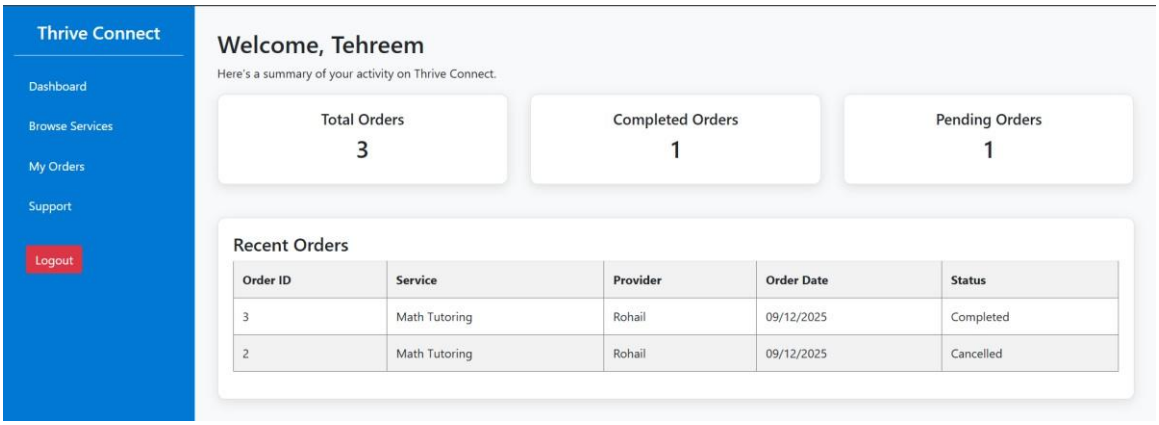
The interface must enforce access control through the roles stored in the database.

This project ensures that students apply complete DBMS principles—from conceptual design to physical implementation—along with practical role-based access management.

Service Provider's Dashboard:



Admin's Dashboard:



Customer's Dashboard: